

top 50 interview questions in django with clear simple understandable terms

Absolutely. Here are **50 important Django interview questions with simple, interview-friendly answers**, especially useful for a **fresher / junior Python-Django developer**.

Top 50 Django Interview Questions & Answers

1. What is Django?

Answer:

Django is a **Python web framework** used to build web applications quickly and securely. It follows the **MVT (Model-View-Template)** architecture.

2. Why is Django popular?

Answer:

Django is popular because it provides many features out of the box, such as:

- Authentication
- Database handling
- URL routing
- Admin panel
- Security features
- Form handling
- Template system

This allows developers to build applications faster.

3. What is MVT in Django?

Answer:

MVT stands for:

- **Model** → Handles database and data
- **View** → Handles business logic and requests
- **Template** → Handles the user interface

Simple flow:

User → URL → View → Model → Database → View → Template → User

4. What is the difference between Django and Flask?

Answer:

Django	Flask
Full-featured framework	Lightweight framework
Built-in ORM	No built-in ORM
Built-in Admin	No built-in Admin
Authentication available	Usually added through extensions
Suitable for large applications	Good for small/simple applications

Interview answer:

"Django is a batteries-included framework, while Flask is lightweight and gives developers more freedom to choose components."

5. What is a Django project?

Answer:

A Django project is the **complete web application**. It contains the main configuration files of the application.

For example:

```
myproject/  
    manage.py  
myproject/  
    settings.py  
    urls.py  
    wsgi.py  
    asgi.py
```

6. What is a Django app?

Answer:

A Django app is a **specific module or feature** inside a Django project.

For example, an HR project could have:

employees

attendance

payroll

leave

One Django project can contain multiple apps.

7. Difference between project and app?

Answer:

Project = Complete website/application.

App = One particular feature or module inside the project.

Example:

HR Management Project

└— Employee App

└— Payroll App

└— Attendance App

└— Leave App

8. What is manage.py?

Answer:

manage.py is a command-line utility used to perform Django project operations.

Examples:

python manage.py runserver

python manage.py makemigrations

python manage.py migrate

python manage.py createsuperuser

python manage.py startapp employees

9. What is settings.py?

Answer:

settings.py contains the **configuration of the Django project**.

It includes:

- Database configuration
 - Installed apps
 - Middleware
 - Templates
 - Static files
 - Security settings
 - Allowed hosts
-

10. What is urls.py?

Answer:

urls.py defines the **URL routes** of a Django application.

Example:

```
path("employees/", views.employee_list)
```

When a user visits:

```
/employees/
```

Django sends the request to the corresponding view.

Models & Database

11. What is a Model in Django?

Answer:

A Model is a Python class that represents a **database table**.

Example:

```
class Employee(models.Model):  
    name = models.CharField(max_length=100)  
    salary = models.IntegerField()
```

Django can create a database table based on this model.

12. What is Django ORM?

Answer:

ORM stands for **Object-Relational Mapping**.

Django ORM allows us to interact with the database using **Python code instead of writing SQL manually**.

Instead of:

```
SELECT * FROM employees;
```

we can use:

```
Employee.objects.all()
```

13. What is a QuerySet?

Answer:

A QuerySet is a collection of objects retrieved from the database.

Example:

```
employees = Employee.objects.all()
```

Here, employees is a QuerySet containing Employee objects.

14. How do you retrieve all records?

Answer:

```
Employee.objects.all()
```

15. How do you retrieve one record?

Answer:

```
Employee.objects.get(id=1)
```

get() is used when we expect **one specific object**.

If the object doesn't exist, it raises an exception.

16. Difference between get() and filter()?

Answer:

get()

Returns **one object**.

```
Employee.objects.get(id=1)
```

filter()

Returns a **QuerySet**, which can contain multiple objects.

```
Employee.objects.filter(department="IT")
```

17. How do you create a record using Django ORM?

Answer:

```
Employee.objects.create(  
    name="Ravi",  
    salary=50000  
)
```

18. How do you update a record?

Answer:

```
employee = Employee.objects.get(id=1)  
employee.salary = 60000  
employee.save()
```

Or:

```
Employee.objects.filter(id=1).update(salary=60000)
```

19. How do you delete a record?

Answer:

```
employee = Employee.objects.get(id=1)  
employee.delete()
```

20. What are migrations in Django?

Answer:

Migrations are used to **apply changes made to Django models to the database**.

For example, if we add:

```
age = models.IntegerField()
```

we create a migration:

```
python manage.py makemigrations
```

Then apply it:

```
python manage.py migrate
```

21. Difference between makemigrations and migrate?

Answer:

makemigrations:

Creates migration files based on model changes.

migrate:

Applies those migration files to the database.

Simple:

Model change → makemigrations → migration file → migrate → database

22. What is a ForeignKey?

Answer:

ForeignKey creates a **many-to-one relationship** between two models.

Example:

```
class Department(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
class Employee(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
department = models.ForeignKey(
    Department,
    on_delete=models.CASCADE
)
```

One department can have many employees.

23. What is OneToOneField?

Answer:

OneToOneField creates a **one-to-one relationship**.

Example:

One Employee → One EmployeeProfile

```
profile = models.OneToOneField(Employee, ...)
```

24. What is ManyToManyField?

Answer:

ManyToManyField represents a many-to-many relationship.

Example:

Student ↔ Courses

One student can have many courses, and one course can have many students.

Views & Templates

25. What is a View in Django?

Answer:

A view contains the **logic that processes a request and returns a response**.

Example:

```
def home(request):
    return render(request, "home.html")
```

26. What is a Function-Based View?

Answer:

A Function-Based View, or FBV, uses a normal Python function.

Example:

```
def home(request):  
    return HttpResponse("Hello")
```

27. What is a Class-Based View?**Answer:**

A Class-Based View uses a Python class to handle requests.

Example:

```
from django.views import View  
  
class HomeView(View):  
    def get(self, request):  
        return HttpResponse("Hello")
```

28. FBV vs CBV?**Answer:****FBV:**

- Simple
- Easy to understand
- Uses functions

CBV:

- Uses classes
- Supports inheritance
- Reusable
- Useful for larger applications

For beginners, FBVs are often easier to understand.

29. What is a Template in Django?

Answer:

A template is usually an **HTML file** containing dynamic Django template syntax.

Example:

```
<h1>Hello {{ name }}</h1>
```

If:

```
name = "Nagesh"
```

the browser displays:

Hello Nagesh

30. What is Django Template Language?

Answer:

Django Template Language, or DTL, allows us to add **dynamic content and logic to HTML templates**.

Examples:

```
{{ name }}
```

```
{% if user.is_authenticated %}
```

```
{% for employee in employees %}
```

31. What is render()?

Answer:

render() combines a template with data and returns an HTTP response.

Example:

```
return render(  
    request,  
    "home.html",  
    {"name": "Nagesh"}  
)
```

32. What is redirect()?

Answer:

redirect() sends the user to another URL.

Example:

```
return redirect("home")
```

It is commonly used after successful form submission.

Forms & Authentication

33. What are Django Forms?

Answer:

Django Forms help us **create, validate, and process HTML forms**.

They reduce the amount of manual form-handling code.

34. Difference between Form and ModelForm?

Answer:

Form

Used when the form isn't directly connected to a database model.

```
class ContactForm(forms.Form):
```

```
    name = forms.CharField()
```

ModelForm

Created from a model and can save data directly to the database.

```
class EmployeeForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Employee
```

```
        fields = "__all__"
```

35. What is form validation?

Answer:

Form validation checks whether the submitted data is **correct and acceptable**.

For example:

- Email must be valid
- Age must be a number
- Required fields cannot be empty

Django provides built-in validation mechanisms.

36. What is CSRF?

Answer:

CSRF stands for **Cross-Site Request Forgery**.

It is a security attack where an attacker tries to make a user's browser perform an unwanted action.

Django protects POST forms using a CSRF token.

Example:

```
<form method="POST">

    {% csrf_token %}

</form>
```

37. What is Django authentication?

Answer:

Django provides a built-in authentication system for:

- User registration
- Login
- Logout
- Password management
- User permissions

It uses Django's built-in User model or a custom user model.

38. What is middleware?

Answer:

Middleware is a layer that processes requests and responses **before they reach the view or after the view returns a response.**

Examples include:

- Authentication middleware
- Session middleware
- CSRF middleware
- Security middleware

Simple flow:

Request



Middleware



View



Middleware



Response

APIs & REST**39. What is an API?****Answer:**

API stands for **Application Programming Interface.**

It allows two software systems to communicate with each other.

For example:

React Frontend



REST API



Django Backend



Database

40. What is Django REST Framework?

Answer:

Django REST Framework, or **DRF**, is a toolkit used to build **Web APIs using Django**.

It provides features such as:

- Serializers
 - API views
 - Authentication
 - Permissions
 - Browsable API
 - Pagination
-

41. What is a Serializer in DRF?

Answer:

A serializer converts complex Python/Django objects into formats such as **JSON**, and also validates incoming JSON data.

Example:

```
class EmployeeSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = Employee
```

```
        fields = "__all__"
```

42. What is JSON?

Answer:

JSON stands for **JavaScript Object Notation**.

It is a common format used for exchanging data between frontend and backend.

Example:

```
{  
  "name": "Nagesh",  
  "age": 22  
}
```

43. What are HTTP methods?

Answer:

Method Purpose

GET	Retrieve data
POST	Create data
PUT	Update complete data
PATCH	Partially update data
DELETE	Delete data

Example:

```
GET /employees/  
POST /employees/  
PUT /employees/1/  
DELETE /employees/1/
```

44. What is a REST API?

Answer:

A REST API is an API architecture that uses HTTP methods and resources to communicate between applications.

Example:

```
GET /employees/  
POST /employees/  
GET /employees/1/
```

PUT /employees/1/

DELETE /employees/1/

Important Advanced Questions

45. What is select_related()?

Answer:

select_related() is used to optimize database queries when working with **ForeignKey** or **OneToOne** relationships.

Example:

```
Employee.objects.select_related("department")
```

It can reduce the number of database queries.

46. What is prefetch_related()?

Answer:

prefetch_related() is used to optimize queries for relationships such as **ManyToMany** and **reverse ForeignKey** relationships.

Example:

```
Student.objects.prefetch_related("courses")
```

47. What is the difference between select_related() and prefetch_related()?

Answer:

select_related()

- Uses SQL JOIN
- Mainly for ForeignKey and OneToOne
- Usually one database query

prefetch_related()

- Performs separate queries and combines results in Python
- Useful for ManyToMany and reverse relationships

Easy way to remember:

select_related = JOIN

prefetch_related = separate query + Python combination

48. What is caching in Django?

Answer:

Caching stores frequently used data temporarily so that Django doesn't have to perform the same expensive operation repeatedly.

It can improve application performance.

Common caching systems include:

- Redis
- Memcached

Example:

Without cache:

User → Django → Database

With cache:

User → Django → Cache

↓

Database only when needed

49. What is DEBUG in Django?

Answer:

DEBUG controls whether Django shows detailed error information.

During development:

DEBUG = True

In production:

DEBUG = False

We should **never use DEBUG=True in production** because it can expose sensitive application information.

50. How does a Django request work?

Answer:

This is one of the **most important interview questions**.

The basic flow is:

User sends request

↓

URL

↓

urls.py

↓

View

↓

Model

↓

Database

↓

View

↓

Template

↓

Response

↓

User

For an API:

Client

↓

URL



View / DRF View



Serializer



Model / ORM



Database



JSON Response



Client

☆ Best interview explanation

"When a user sends a request, Django first checks the URL patterns. The matching URL calls the appropriate view. The view processes the business logic and can interact with the database through Django ORM. If it is a web page, the view sends data to a template, which generates the HTML response. If it is an API, the data is usually serialized into JSON and returned to the client."

☆ 10 Django Questions You Should Prepare First

If you're preparing for a **Django fresher/junior interview**, prioritize these:

1. **What is Django?**
2. **Explain MVT architecture.**
3. **Project vs App**
4. **Explain Django request-response cycle.**
5. **What is Django ORM?**
6. **What are migrations?**
7. **get() vs filter()**
8. **Function-Based View vs Class-Based View**

9. **What is Django REST Framework?**

10. **Explain your Django project end-to-end.**

For your interviews, I would especially prepare **project-based Django questions** next: interviewers often take your resume project and ask *“Why Django?”*, *“Explain your models”*, *“How did your API work?”*, *“How did authentication work?”*, *“How did you connect the database?”*, *“What errors did you face and how did you fix them?”*